TREES AND THEIR DEGREES

MARIE-CAMILLE DELARUE

Contents

1. Introduction

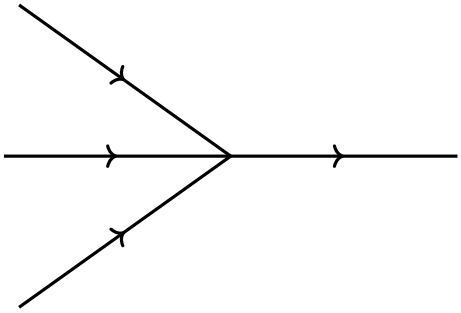
In the course of our study of the family of Higman–Thompson groups, there arose a question that was easy to answer in small dimensions and much harder in higher dimensions. The fundamental reason for this is that it was easy to come up with a reasonable visualization in low dimensions, and much harder in higher dimensions because of the limitations of 3-dimensional space. Therefore, we tried to come up with a good visualization in higher dimensions.

The following document explains this problem and the framework we came up with to answer it in higher dimensions. Reading the following document is recommended to understand the specific question we were working on as well as its implementation.

2. A question about trees

We will consider finite rooted trees, and we call an **a-junction** a vertex of the tree with one parent and a children: For instance, $\curlyvee$ is a 3-junction.

2.1. **Trees with simple junctions.** Let us start by considering the following problem. Consider a finite rooted tree which only has a-junctions, for a an integer. We will think of trees as being oriented from the leaves to the root. The following is an illustration of an (oriented) junction:



Date: September 11, 2026.

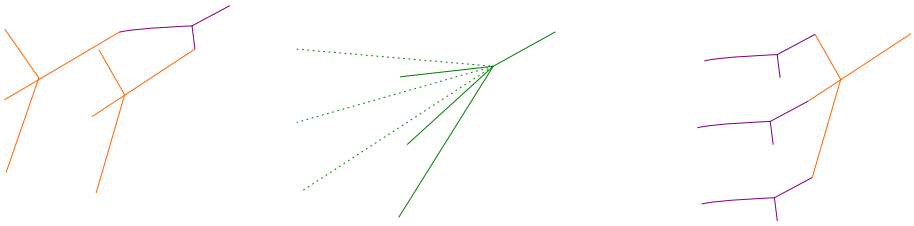
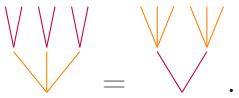


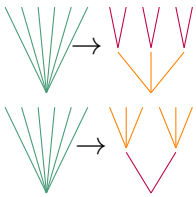
FIGURE 1. The tree is parametrized by $(x, t) \in [0, 1]^N \times [0, 1]$. The barycenter of the junctions is governed by x and the shape of the tree (including how far apart the junctions are) is governed by t.

We will call **degree** of a junction the number of incoming edges minus the number of outgoing edges at the junction. For any a-junction, this number is clearly $a-1$. If the trees have two or more types of junctions, for instance a-junctions and b-junctions, the a-junctions have degree $a - 1$, and the b-junctions have degree $b - 1$.

2.2. **Trees with double junctions.** Now let us consider a family of trees with both a-junctions and b-junctions, for a and b two different integers, with the following additional **interchange relation**:



This means that we identify certain trees even if they do not have the same shape. How can we represent this equality topologically? In topology, we can always relax an equality between two objects by creating a path between them. We add an ab-junction which represents this path:



If our tree lives in N-dimensional space, we are adding an extra dimension which represents time. The position of t in $[0, 1]$ indicates what the form of the tree is in a continuous way as depicted in figure 1. Now our tree is parametrized by an element in $[0, 1]^{N+1}$.

In the program, there is a function `make_family` which holds the combinatorial information about the tree, and a function `tree_at` which to a point in $[0, 1]^{N+1}$

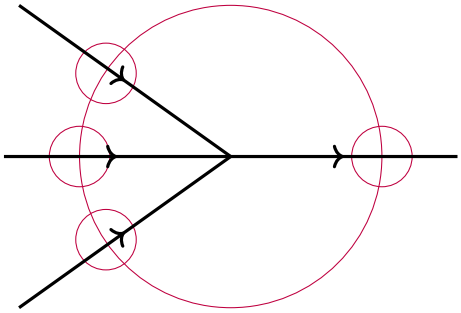


FIGURE 2. The case of a simple junction

and the combinatorial information of the tree associates the corresponding embedding of a tree inside a unit cube $[0,1]^N$.

2.3. **Goal – notion of degree.** We would like to determine the degree of this extra junction. It is tempting to say that the degree of this junction is $ab-1$. However, we now have equivalent trees that do not have the same structure, so how can one determine the degree in this case?

Let's give another definition of the degree by going back to the case of a simple junction. We represent the junction living inside a small neighborhood, depicted as a circle around it in figure **??**. Then imagine a smaller lens traveling around the boundary of the small neighborhood and recording what it sees. The lens will see a single path $a+1$ times: a time with positive orientation and once with negative orientation, and so we recover a total degree of $a-1$.

We will find the degree of the ab-junction by traveling around it and recording what we see. We need to count the number of a-junctions and b-junctions that we see around it. This is how we will construct the detector map.

3. Defining the detector map

We create a detector for a-junctions, and one for b-junctions, according to the following procedure. Given a specific embedding of a tree inside a cube, we look at a smaller detector cube centered inside the unit cube and we record what it contains:

- if it contains a single a-junction and no other connected component of the tree, we record the position of the a-junction, which is a point inside the detector cube, and rescale it to a point of the unit cube.
- if it contains anything else, we record "nothing", which we set as the basepoint.

Therefore, the range of the detector map is a unit cube $[0,1]^N$, whose boundary has been collapsed to a point, which we call the basepoint.

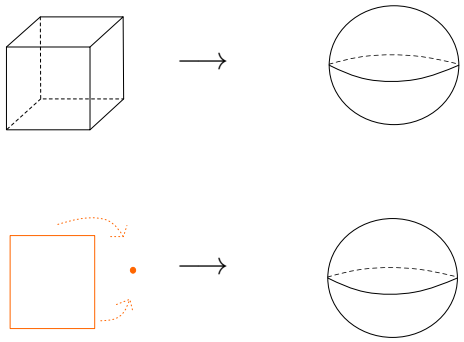


FIGURE 3. The boundary of the 3-dimensional cube and a 2-square whose boundary is glued onto a point are both models for the 2-sphere.

The detector map that we define in the program is actually a composition of `tree_at` and the aforementioned procedure:

- Take a point in the cube $[0,1]^{N+1}$ that we can write (x,t) where $x \in [0,1]^N, t \in [0,1]$;
- Construct an embedding of a tree inside the cube $[0,1]^N$. The barycenter of the junctions is at x and the shape of the tree is determined by t;
- Look inside a smaller detector cube and record what you see;
- Get a point in $([0,1]^N)/(\partial[0,1]^N)$.

This is what the functions `detect_a` and `detect_b` correspond to.

4. Degree of a map between spheres

The detector map is defined from $[0,1]^{N+1}$ to $[0,1]^N/\partial[0,1]^N$. If we restrict the source to the boundary of $[0,1]^{N+1}$, then the source is equivalent to an N-sphere. The range $[0,1]^N/\partial[0,1]^N$ is a cube whose boundary is collapsed to a point, which is also equivalent to an N-sphere, as illustrated in figure **??**. Therefore, the detector map is equivalent to a function from a sphere S^N to itself.

The notion of degree we defined in section 2 is closely related to the notion of degree of maps between spheres. Basically, the degree of a map from a sphere S^N to itself is the number of times the map goes around the target sphere, up to homotopy. It is clearer to see this on an example.

Example 4.1. Imagine the function from the circle S^1 to another S^1 which wraps the circle around itself twice, as illustrated in figure **??**: The degree of this map is 2.

Examples on spheres of higher dimension are more difficult to understand than in dimension 1, but they work in the same way.

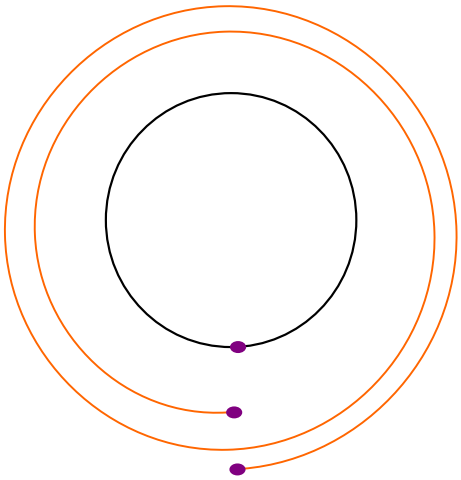


FIGURE 4. The orange circle wraps twice around the first one, starting and ending at the purple point.

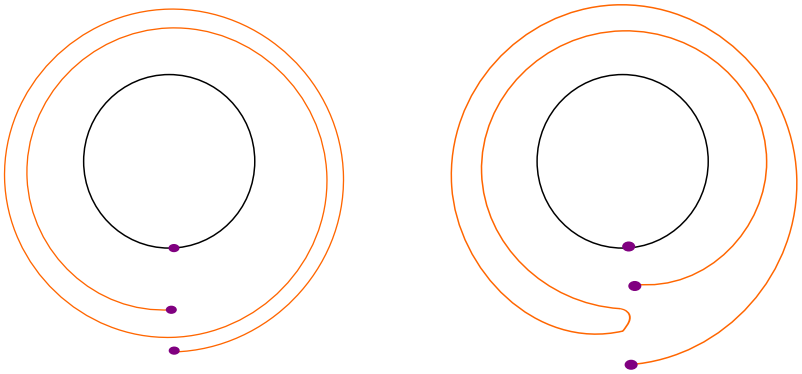


FIGURE 5. The left map is the one introduced earlier. The map on the right wraps the orange circle around the black circle once, then changes directions and wraps around it once more.

4.1. **Degree formula.** If the map is C^1, we can obtain the degree by picking a regular point y in the image (i.e. a point with nonsingular differential), and counting its preimages:

$$d = \sum_{p \in f^{-1}(y)} s_p$$

where s_p is the sign of the Jacobian of f at point p.

Example 4.2. Consider the two maps from S^1 to S^1 depicted in figure **??**: If one were simply to count the preimages at any regular value – for instance, not at the basepoint of the circle (the purple point), one would see that for both maps, there

are two points of the orange circle mapped to the same point of the black circle. However, the degree of the left map is 2, and the degree of the right map is 0. This is because in the first instance, both preimages contribute the same sign to the formula: they are oriented in the same way. In the second instance, they have opposite orientation.

4.2. **Algorithm.** Let us now go through the steps of the algorithm:

-

4.3. **Picking a regular value.** It is very important to pick a good regular value to carry out the above procedure.